Practicum: Open-loop regeling van een aeropendulum

Leerdoelen

- ❒ Je kunt de dynamische respons van een simpel open-loop systeem experimenteel bepalen.
- ❒ Je kunt overweg met de Arduino IDE en begrijpt de werking van een H-brug, PWM en een hoeksensor.
- ❒ Je kunt vanuit Python de Arduino aansturen, meetgegevens opslaan en deze vervolgens analyseren en visualiseren.

Veiligheid

1. Houd tijdens de metingen voldoende afstand van de draaiende propeller en raak deze nooit aan wanneer de voeding is ingeschakeld.
2. Controleer vóór iedere meting of het aeropendulum vrij kan bewegen en of zich geen objecten binnen het bewegingsbereik bevinden.
3. Twijfel je over een handeling of apparaat? Vraag dit altijd aan de practicumbegeleider.

Introductie

Regeltechniek speelt een belangrijke rol binnen de technische natuurkunde. Veel technische systemen, zoals drones, robots, satellieten, productieprocessen en autonome voertuigen, moeten automatisch bestuurd worden om een gewenste positie of toestand te bereiken. Voordat een regelsysteem ontworpen kan worden, is het belangrijk om eerst het gedrag van het systeem zelf goed te begrijpen. Dit gebeurt vaak door het systeem in open-loop te bestuderen.

Bij een open-loop systeem wordt een ingangssignaal aangeboden zonder dat de uitgang wordt teruggekoppeld naar de regeling. De besturing corrigeert dus niet voor verstoringen of afwijkingen. Hoewel deze manier van regelen relatief eenvoudig is, vormt zij de basis voor vrijwel alle moderne regelsystemen. Door de respons van een systeem op verschillende ingangssignalen te meten, kunnen eigenschappen zoals de gevoeligheid, tijdconstante, demping en niet-lineariteiten worden bepaald.

Een eenvoudig voorbeeld van open-loop regeling is een magnetron. Je stelt bijvoorbeeld een vermogen van 800 W en een tijd van 2 minuten in. De magnetron voert deze instelling uit, maar meet niet of het eten daadwerkelijk de gewenste temperatuur heeft bereikt. Is het eten na 2 minuten nog te koud? Dan corrigeert de magnetron dit niet automatisch. De uitgang (temperatuur van het eten) wordt namelijk niet teruggekoppeld naar de ingang (vermogen en verwarmingstijd).

In dit practicum onderzoek je het dynamische gedrag van een aeropendulum. Dit systeem bestaat uit een arm die om een draaipunt kan roteren en wordt aangedreven door een elektrische propeller. De propeller levert een stuwkracht die de arm omhoog beweegt, terwijl de zwaartekracht de arm terug naar beneden probeert te brengen. Door de motorspanning stapsgewijs te variëren ontstaat een evenwicht tussen deze twee krachten, waardoor verschillende stationaire hoeken en dynamische responsen kunnen worden bestudeerd.

Tijdens het practicum meet je de hoek van het aeropendulum als functie van de tijd voor verschillende motorspanningen. De meetgegevens worden vervolgens in geanalyseerd om de dynamische eigenschappen van het systeem te bepalen en te vergelijken met een eenvoudig fysisch model. De kennis die in dit practicum wordt opgedaan vormt de basis voor een vervolgpracticum, waarin hetzelfde systeem met behulp van terugkoppeling (*closed-loop control*) automatisch zal worden geregeld.

Theorie

Een aeropendulum bestaat uit een draaibare arm met aan het uiteinde een elektromotor en propeller. De propeller levert een stuwkracht waardoor de arm omhoog beweegt. De zwaartekracht werkt deze beweging tegen. Door het vermogen van de motor te variëren kan de positie van het aeropendulum worden beïnvloed.

In dit practicum wordt het systeem in open-loop onderzocht. Dit betekent dat het vermogen van de motor vooraf wordt ingesteld en niet automatisch wordt aangepast op basis van de gemeten hoek. Voor iedere instelling wordt vervolgens onderzocht hoe het aeropendulum reageert.

De propeller levert een stuwkracht F_T op een afstand l_T van het draaipunt. Het bijbehorende moment is

$$M_T = l_T F_T. \tag{1}$$

Tegelijkertijd oefent de zwaartekracht een moment uit op het systeem. Wanneer het zwaartepunt zich op een afstand l van het draaipunt bevindt, is dit moment

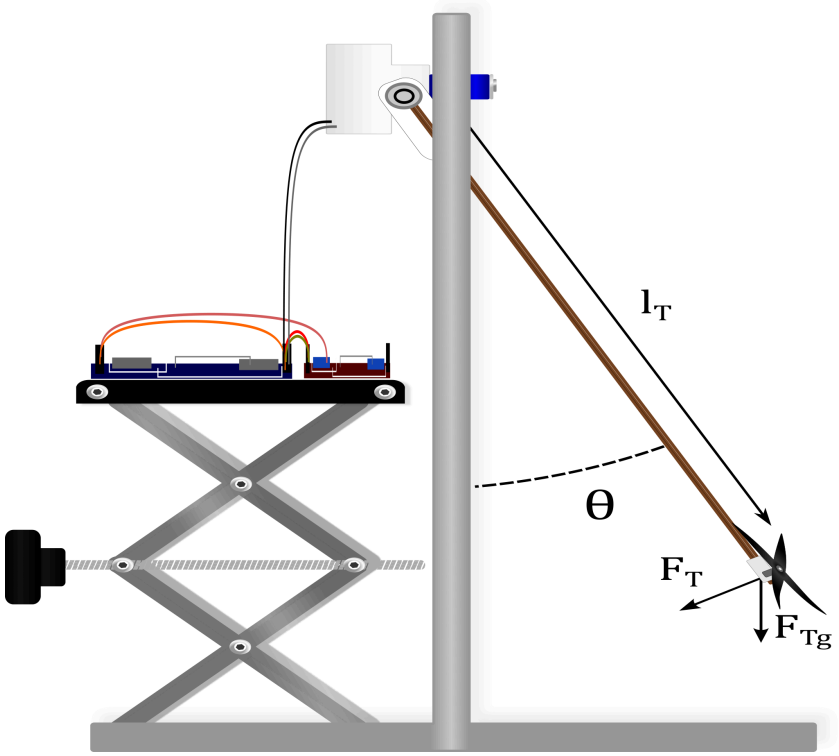
$$M_g = mgl \sin\theta, \tag{2}$$

waarbij m de massa van het bewegende systeem is, g de zwaartekrachtsversnelling en θ de hoek van de arm.

Volgens de tweede wet van Newton voor rotatie wordt de beweging bepaald door de som van deze momenten:

$$J\ddot{\theta} = l_T F_T - mgl \sin\theta, \tag{3}$$

waarbij J het massatraagheidsmoment van het aeropendulum rond het draaipunt is.



Figuur 1: Schematische weergave van het aeropendulum en de relevante krachten.

De stuwkracht van de propeller wordt bepaald door het vermogen dat aan de motor wordt geleverd. Het elektrisch ingangsvermogen is

$$P_{\text{el}} = UI, \tag{4}$$

met U de spanning over de motor en I de stroom door de motor. Slechts een deel van dit vermogen wordt door de motor en propeller omgezet in mechanisch vermogen. Met een rendement η kan dit worden geschreven als

$$P_{\text{motor}} = \eta UI. \tag{5}$$

De stuwkracht kan daarom worden beschouwd als een functie van het geleverde vermogen,

$$F_T = f(P_{\text{motor}}). \tag{6}$$

De precieze vorm van deze relatie hangt af van onder andere de motor en de propeller en hoeft niet lineair te zijn.
Wanneer het aeropendulum uiteindelijk tot stilstand komt, is de hoekversnelling nul. Het moment van de propeller is dan gelijk aan het moment van de zwaartekracht:

$$l_T F_T = mgl \sin\theta. \tag{7}$$

Hieruit volgt

$$\theta = \arcsin\left(\frac{l_T F_T}{mgl}\right). \tag{8}$$

Een groter motorvermogen leidt in het algemeen tot een grotere stuwkracht en daarmee tot een grotere evenwichtshoek. Tijdens het practicum wordt deze relatie experimenteel onderzocht door het motorvermogen te variëren en de resulterende hoek van het aeropendulum te meten.

Componenten

Het aeropendulum bestaat uit verschillende elektronische en mechanische componenten. De Arduino stuurt de motor aan via een H-brug en leest tegelijkertijd de hoek van het pendulum uit met een magnetische hoeksensor. De aansluittabel bij opdracht 0 geeft een overzicht van de gebruikte aansluitingen.

Arduino Uno R4 Minima

Een Arduino is een programmeerbaar microcontrollerbord waarmee elektronische componenten kunnen worden aangestuurd en sensoren kunnen worden uitgelezen. In dit practicum wordt een **Arduino Uno R4 Minima** gebruikt.
De Arduino voert continu een programma uit waarin twee taken centraal staan: het aansturen van de elektromotor en het meten van de hoek van het aeropendulum. Hiervoor beschikt de Arduino onder andere over digitale aansluitingen, analoge ingangen en verschillende communicatieprotocollen.
In deze opstelling worden de digitale pinnen D9, D10 en D11 gebruikt voor de aansturing van de motor. De AS5600-hoeksensor communiceert via de pinnen A4 en A5 met de Arduino.

H-brug

Een elektromotor kan niet rechtstreeks vanuit een digitale uitgang van de Arduino worden gevoed. De motor vraagt aanzienlijk meer stroom dan een uitgang van de microcontroller kan leveren. De Arduino levert daarom alleen het *stuursignaal*; het elektrische vermogen voor de motor wordt geleverd door de externe voeding.

Tussen de Arduino en de motor bevindt zich daarom een **L298N H-brug**. Deze schakeling kan de externe voedingsspanning naar de motor schakelen. De naam H-brug komt van de configuratie van de interne schakelaars, die schematisch de vorm van de letter H heeft.

Door verschillende schakelaars te openen en sluiten kan de stroom door de motor in beide richtingen lopen. Daarmee kan ook de draairichting van de motor worden veranderd. Hiervoor gebruikt de L298N de ingangen IN1 en IN2. In onze opstelling worden deze aangestuurd door D10 en D11 van de Arduino.

Pulse Width Modulation

Naast de draairichting willen we ook kunnen regelen hoeveel vermogen de motor ontvangt. Hiervoor wordt **Pulse Width Modulation (PWM)** gebruikt.

Bij PWM wordt de voedingsspanning niet continu veranderd. In plaats daarvan wordt de spanning snel aan- en uitgeschakeld. De verhouding tussen de tijd dat het signaal aan staat en de totale periodetijd noemen we de *duty cycle* D:

$$D = \frac{t_{\text{aan}}}{T}. \tag{9}$$

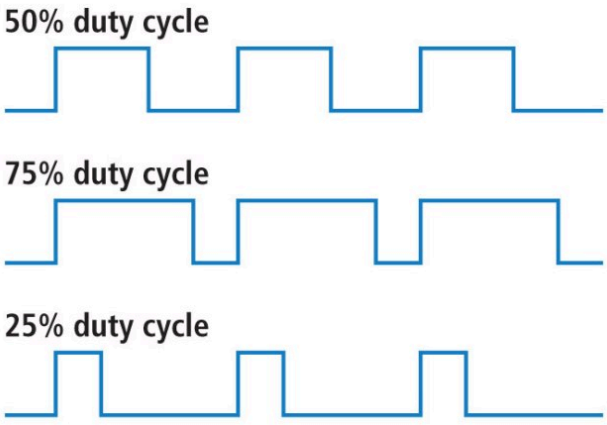
Een duty cycle van 100% betekent dat de motor continu wordt aangestuurd. Bij 50% staat de spanning gedurende de helft van iedere periode aan en bij 0% staat de motor volledig uit.

Voor een ideale PWM-schakeling is de gemiddelde spanning over één periode

$$U_{\text{gem}} = D\,U_{\text{voeding}}. \tag{10}$$

De motor reageert door zijn mechanische en elektrische traagheid niet afzonderlijk op iedere snelle puls. Daardoor kan met de duty cycle het gedrag van de motor worden geregeld.

In onze opstelling genereert pin D9 van de Arduino het PWM-signaal. Dit signaal wordt aangesloten op de **ENA**-ingang van de L298N. De Arduino bepaalt dus wanneer de H-brug de motor inschakelt, terwijl het daadwerkelijke motorvermogen uit de externe voeding komt.



Figuur 2: Voorbeeld van PWM-signalen met verschillende duty cycles. De amplitude blijft gelijk, maar de verhouding tussen de aan- en uittijd verandert.

AS5600-hoeksensor

De hoek van het aeropendulum wordt gemeten met een **AS5600 magnetische hoeksensor**. Op de draaiende as van het aeropendulum wordt een magneet geplaatst. De sensor meet de richting van het magnetische veld en kan daaruit de hoek van de as bepalen. Omdat er geen mechanisch contact nodig is tussen de sensor en de draaiende as, treedt hierbij vrijwel geen extra mechanische weerstand op.

De AS5600 heeft een resolutie van 12 bit. Eén volledige omwenteling wordt daardoor verdeeld in

$$2^{12} = 4096 \tag{11}$$

mogelijke waarden. Een ruwe sensorwaarde kan worden omgerekend naar een hoek volgens

$$\theta = \frac{\text{sensorwaarde}}{4096}\, 360^\circ. \tag{12}$$

De sensor wordt digitaal uitgelezen via het **I²C-protocol**. Hiervoor zijn twee verbindingen nodig: SDA (*Serial Data*) voor het versturen van gegevens en SCL (*Serial Clock*) voor het kloksignaal. In deze opstelling is SDA aangesloten op A4 en SCL op A5 van de Arduino.
De positie van de magneet ten opzichte van de sensor is belangrijk. Wanneer de magneet te ver van de sensor staat of niet goed boven het midden van de sensor is geplaatst, kan de hoekmeting onbetrouwbaar worden. Controleer daarom voor iedere meting of de sensor correct functioneert. Wanneer de arm langzaam met de hand wordt bewogen, moet de gemeten hoek vloeiend veranderen. Een constante waarde, grote sprongen of een foutmelding kan betekenen dat de magneet niet correct ten opzichte van de sensor staat.

Voorbereiding

1 Bekijk de geometrie van het aeropendulum. Onder welke voorwaarde geldt $l = l_T$? Leg in woorden uit wat deze aanname fysisch betekent.

2 In de stationaire toestand staat de arm stil en zijn de momenten rond het draaipunt in evenwicht. Laat zien dat dan geldt:

$$l_T F_T = mgl \sin\theta. \tag{13}$$

3 De motor zet niet al het opgenomen elektrische vermogen om in mechanisch vermogen. Neem een rendement van $\eta = 0.70$ aan:

$$P_{\text{mech}} = \eta U I. \tag{14}$$

Bereken bij een voedingsspanning van $U = 4$ V het mechanisch vermogen voor een aantal verschillende motorstromen.

4 Neem aan dat de stuwkracht van de propeller evenredig is met het mechanisch vermogen:

$$F_T = k_P P_{\text{mech}}, \tag{15}$$

waarbij k_P een constante is die de eigenschappen van de motor en propeller beschrijft. Combineer de bovenstaande vergelijkingen en leid een uitdrukking af voor de stationaire hoek θ als functie van de motorstroom I.

5 Kies realistische waarden voor m, l, l_T en k_P. Licht kort toe waarom je deze waarden hebt gekozen. Maak vervolgens in Desmos een grafiek van θ als functie van I.

6 Voorspel op basis van je grafiek wat er gebeurt wanneer de motorstroom wordt verhoogd. Verwacht je dat het eenvoudige model voor alle motorstromen even goed werkt? Leg kort uit.

Experiment

Werk de opdrachten in onderstaande volgorde uit. Open iedere sketch vanuit zijn eigen gelijknamige map onder `Arduino/OpenLoop`. Selecteer in de Arduino IDE de Arduino Uno R4 Minima en de juiste poort. Installeer de bibliotheek `AS5600` van Rob Tillaart. Alle drie de scripts gebruiken 115200 baud.

Opdracht 0: opbouw van de opstelling en aansluitschema

1. Zet de Arduino Uno R4 Minima, AS5600-hoeksensor, L298N H-brug, motor en externe voeding klaar. Bekijk hoe de arm en de magneet bij het draaipunt zijn gemonteerd. Controleer of de arm vrij kan bewegen.
2. Bouw de opstelling op met uitgeschakelde externe voeding en losgekoppelde USB-kabel. Sluit de hoeksensor en de stuuringangen van de H-brug aan volgens tabel 1. Controleer per draad de Arduino-pin, draadkleur en aansluiting op de component.
3. Laat de aansluiting van motor en de externe voeding op de H-brug aansluiting controleren door de practicumbegeleider. Spreek de voedingsspanning en stroomlimiet af en noteer deze in je labjournaal.
4. Laat de complete opstelling controleren voordat je USB aansluit. Houd de motorvoeding uitgeschakeld: in opdracht 1 test je eerst alleen de hoeksensor met `Stap_1_Sensor.ino`.

Tabel 1: Aansluitingen van de Arduino naar de AS5600-hoeksensor en de L298N H-brug.

Van Arduino	Draadkleur	Naar component	Aansluiting
GND (Analog)	Zwart	Hoeksensor	GND
3.3 V	Rood	Hoeksensor	VCC
A4	Groen	Hoeksensor	SDA
A5	Wit	Hoeksensor	SCL
D9 (digitaal, PWM)	Paars	H-brug	ENA
D10 (digitaal)	Oranje	H-brug	IN1
D11 (digitaal)	Geel	H-brug	IN2

Opdracht 1: hoeksensor uittesten met stap 1

1. Gebruik de in opdracht 0 opgebouwde en gecontroleerde opstelling. Laat de motorvoeding uitgeschakeld en sluit alleen USB aan op de Arduino. Open `Arduino/OpenLoop/Stap_1_Sensor/Stap_1_Sensor.ino`: deze sketch leest uitsluitend de hoeksensor uit en stuurt de motor niet aan.
2. Upload het script en open Serial Monitor of Serial Plotter op 115200 baud. Het script geeft iedere 100 ms een regel zoals `Hoek:90.000` weer.
3. Bekijk de omzetting `sensor.readAngle() * 360.0 / 4096.0` in de code. Leg uit waarom door 4096 wordt gedeeld en bepaal hoeveel graden een sensorstap voorstelt.
4. Beweeg de arm langzaam met de hand. Controleer of de gemeten hoek vloeiend verandert en noteer de sensorhoek bij de verticale ruststand en bij enkele bekende armstanden. Onderzoek in welke richting de sensorhoek toeneemt en waar de overgang tussen 359 en 0 graden ligt.
5. Leg vast hoe je de ruwe sensorhoek omzet naar de werkhoek θ uit het theoretische model, met nul bij de verticale ruststand. Voor deze sensormontage geldt `werkhoek = sensorhoek - beginhoek`, waarbij `beginhoek` de ruwe sensorhoek bij de verticale ruststand is. Vouw het verschil terug naar −180 tot 180 graden: trek bij een waarde boven 180 graden 360 graden af en tel bij een waarde onder −180 graden 360 graden op. De scripts van stap 1 tot en met 3 leveren de ruwe sensorhoek; de hoekoffset wordt bij de verwerking toegepast.
6. Bewaar je waarnemingen in het labjournaal. Laat een constante waarde, onverwachte sprongen of een onjuiste hoek eerst controleren voordat je de motor aansluit. Deze scripts controleren de sensorstatus niet automatisch.

Opdracht 2: vaste PWM en stationaire hoek

1. Open `Stap_2_PWM_en_sensor/Stap_2_PWM_en_sensor.ino`. Schakel de voeding uit en controleer de in opdracht 0 aangesloten H-brug en motor. Gebruik de aansluittabel voor D9, D10 en D11. Laat de motorvoeding, de gemeenschappelijke massa en de ENA-aansluiting controleren door de begeleider.
2. Zoek `pwm_waarde` en de drie motorpinnen op in de code. Leg uit hoe D9 het PWM-signaal levert en hoe D10 en D11 de draairichting vastleggen. Bereken de duty cycle bij de ingestelde waarde met $D = \frac{\text{pwm_waarde}}{255}$.
3. Spreek met de begeleider een veilig PWM-bereik, voedingsspanning en stroomlimiet af. Het codebereik 0-255 is geen vastgesteld veilig werkbereik. Begin met `pwm_waarde = 0` en upload met uitgeschakelde motorvoeding.
4. Kies vervolgens een afgesproken PWM-waarde en upload opnieuw met de motorvoeding uit. Houd afstand en schakel daarna de motorvoeding in. De sketch stuurt de motor direct na uploaden of resetten aan met `pwm_waarde`. Ook het openen van een seriele verbinding kan een reset veroorzaken. Er is geen startcommando, automatische meetduur of softwarematige stop in dit script; stop door de motorvoeding uit te schakelen.
5. Bekijk de hoek in Serial Monitor of Serial Plotter. Wacht tot de arm een stationaire positie bereikt, of noteer dat dit niet gebeurt. Noteer de PWM-waarde, duty cycle, stationaire werkhoek, voedingsspanning en motorstroom.
6. Herhaal dit voor meerdere PWM-waarden binnen het afgesproken bereik, met minimaal drie proeven per instelling. Schakel voor iedere wijziging of upload de motorvoeding uit. Houd voedingsspanning en stroomlimiet gelijk.
7. Maak een grafiek van de gemiddelde stationaire werkhoek tegen de PWM-waarde, met foutbalken op basis van de herhalingen. Bepaal de dode zone en onderzoek of de relatie lineair is. Dit script verstuurt alleen `Hoek:`-regels, zonder tijdstempels of CSV-kop.

Opdracht 3: aansturen en meten met Python

1. Open `Stap_3_Python_aansturing/Stap_3_Python_aansturing.ino` en upload met uitgeschakelde motorvoeding. De motor begint bij deze sketch met PWM 0. Sluit Serial Monitor en Serial Plotter voordat Python dezelfde poort opent.
2. Installeer in je Python-omgeving `pyserial`, `pandas` en `matplotlib`. Gebruik het voorbeeld hieronder en vul de juiste poort en een met de begeleider afgesproken PWM-waarde in. Start eerst een proef met PWM 0 en de motorvoeding uit om communicatie en hoekmeting te controleren.
3. Bestudeer de commando's in de tabel. Elk commando eindigt met een regeleinde. `START` schakelt alleen de meetuitvoer in; `PWM:` stelt de motor afzonderlijk in, ook als er niet wordt gemeten. Verstuur tijdens de proef regelmatig `PING`. Zonder heartbeat of ander commando dat de watchdog bijwerkt stopt de sketch na ruim 3 seconden de motor en de meetuitvoer.
4. Schakel na de communicatiecontrole de gecontroleerde motorvoeding in en voer een proef uit met de afgesproken PWM. Houd de fysieke uitschakeling bereikbaar. De sketch heeft geen automatische hoek- of sensorbewaking. Het Python-voorbeeld stopt na 30 seconden en verstuurt ook bij een onderbreking `STOP`. Schakel na afloop de motorvoeding uit.
5. Sla iedere proef op onder een eigen bestandsnaam. De Arduino verstuurt meetregels als `tijd_in_ms,hoek_in_graden`, zonder kopregel. De tijd loopt vanaf het starten van de Arduino, niet vanaf `START`. Regels die beginnen met `DEBUG_` zijn geen meetpunten; het Python-voorbeeld herkent deze en schrijft zelf de CSV-kop.
6. Herhaal de metingen minimaal drie keer per PWM-instelling. Noteer de PWM-waarde, voedingsspanning, motorstroom, beginhoek en hoekconversie apart. Maak grafieken van de werkhoek tegen de tijd en bepaal per proef de gemiddelde stationaire hoek en standaarddeviatie over een onderbouwd gekozen tijdvenster.
7. Vergelijk de stationaire werkhoek met de resultaten van opdracht 2 en het theoretische model. Gebruik de gemeten motorstroom voor vergelijking met het model als functie van stroom; een PWM-waarde is geen gemeten spanning of stroom. Bespreek de dode zone, oscillaties, niet-lineariteit en meetonzekerheid.

Commando	Werking in script 3
`PWM:60`	Stelt de PWM-waarde in (codebereik 0-255) en vernieuwt de heartbeat.
`DT:100`	Stelt het meetinterval in milliseconden in (10-60000 ms).
`START`	Start de meetuitvoer en vernieuwt de heartbeat.
`PING`	Vernieuwt de heartbeat; verstuur bijvoorbeeld iedere 0,5 seconde.
`STOP`	Stopt de meetuitvoer en zet de PWM op 0.

Aansturen en opslaan vanuit Python

```
import csv
import time
import serial

from serial.tools import list_ports

print('Beschikbare poorten:', [(p.device, p.description) for p in
list_ports.comports()])
poort = 'COM4'  # Vul de poort uit de Arduino IDE in.
pwm = 0         # Eerst testen op 0; daarna afgesproken waarde.
meetduur_s = 30
bestandsnaam = f'ol_pwm{pwm}_run1.csv'

with serial.Serial(poort, 115200, timeout=0.1) as arduino:
    try:
        time.sleep(2)  # Wacht op een eventuele reset.
        arduino.reset_input_buffer()
        print(f'Verbonden met {poort}; ingestelde PWM: {pwm}')
        if pwm == 0:
            print('PWM is 0: de motor blijft uit tijdens deze
communicatieproef.')

        def stuur(tekst):
            arduino.write((tekst + '\n').encode('ascii'))
            arduino.flush()

        stuur('PWM:0')
        stuur('DT:100')
        with open(bestandsnaam, 'w', newline='') as bestand:
            schrijver = csv.writer(bestand)
            schrijver.writerow(['t_ms', 'angle_deg'])
            stuur('START')
            stuur(f'PWM:{pwm}')
            begin = time.monotonic()
            laatste_ping = begin
            laatste_meetpunt = begin
            aantal = 0
            print('Meting gestart; onderbreek met Ctrl+C om te stoppen.')
            while time.monotonic() - begin < meetduur_s:
                nu = time.monotonic()
                if nu - laatste_ping >= 0.5:
                    stuur('PING')
                    laatste_ping = nu
                regel = arduino.readline().decode('ascii').strip()
                if not regel:
                    if time.monotonic() - laatste_meetpunt > 3:
                        raise TimeoutError(
                            'Geen meetgegevens: controleer poort, baudrate en
of '
                            'Stap_3_Python_aansturing is geupload. Sluit
Serial Monitor.'
                        )
                    continue
                if regel.startswith('DEBUG_'):
                    print(regel)
                    if regel == 'DEBUG_WATCHDOG':
                        raise RuntimeError('Watchdog: proef gestopt')
                    continue
                velden = regel.split(',')
                if len(velden) == 2:
```

Meetgegevens inlezen

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(bestandsnaam)
if df.empty:
    raise ValueError('Geen meetpunten opgeslagen')
df['t_s'] = (df['t_ms'] - df['t_ms'].iloc[0]) / 1000
print(df.keys())
print(df['t_ms'].diff().describe())
beginhoek = 90.0  # Vul de gemeten ruwe sensorhoek in rust in.
df['werkhoek_deg'] = (df['angle_deg'] - beginhoek + 180) % 360 - 180
plt.plot(df['t_s'], df['werkhoek_deg'])
plt.xlabel('Tijd sinds eerste meetpunt (s)')
plt.ylabel('Werkhoek (graden)')
plt.grid()
plt.show()
```

De meetfrequentie is bij `DT:100` nominaal 10 Hz. Controleer de werkelijke intervallen met de tijdstempels. Het eerste meetpunt ligt na het startcommando; meet de beginhoek vooraf bij PWM 0 als je de respons op een PWM-stap wilt onderzoeken.

Afronding

1. Laat de meetresultaten controleren door de practicumbegeleider.
2. Voeg alle grafieken toe aan je digitale labjournaal en geef iedere figuur een duidelijk onderschrift.
3. Vul de sectie **Resultaten en Discussie** aan. Beschrijf hierin de relatie tussen de Arduino-uitgang en de stationaire hoek en vergelijk de experimentele resultaten met het theoretische model.
4. Reflecteer op de beperkingen van een open-loop systeem en beschrijf waarom terugkoppeling in veel technische toepassingen gewenst is.